

Resolución del Práctico 2

Testing de Software 2024

Índice

Introducción	3
1. Ejercicio 1	3
1.1. Lectura	3
2. Ejercicio 2	4
2.1. Objetivo	4
2.2. Estrategia de parametrización	4
2.3. Casos elegidos y modelo RIPR	4
2.4. Implementación abreviada	4
2.5. Resultado sobre la versión defectuosa	5
2.6. Cómo ejecutar	6
3. Ejercicio 3	6
3.1. Objetivo	6
3.2. Modelo de la clase	6
3.3. Implementación de <code>rep0k</code>	6
3.4. Correcciones sobre la implementación	7
3.5. Suite de tests	7
3.6. Tests de <code>rep0k</code> y uso de reflexión	8
3.7. Inconvenientes encontrados	8
3.8. Cómo ejecutar	9
4. Ejercicio 4	9
4.1. Objetivo	9
4.2. Estructura de los archivos CSV	9
4.3. Mapeo CSV → ejecución	9
4.4. Tests parametrizados	10

4.5. Tests negativos adicionales	10
4.6. Cómo ejecutar	10
5. Ejercicio 5	11
5.1. Objetivo	11
5.2. Contexto - el Zune bug	11
5.3. Versión original	11
5.4. Diseño de los tests parametrizados	11
5.5. Fallas detectadas	12
5.6. Corrección aplicada	12
5.7. Cómo ejecutar	13
6. Ejercicio 6	13
6.1. Objetivo	13
6.2. Modelo de la clase	13
6.3. Invariante de representación	13
6.4. Implementación	13
6.5. Tests parametrizados	14
6.6. Cómo ejecutar	15
Conclusiones	16

Introducción

Este documento reúne la resolución del *Práctico 2* de *Testing de Software 2024*. La consigna profundiza en testing automatizado con JUnit 5, en particular en el enfoque *data-driven* a través de la anotación `@ParameterizedTest` y sus distintas fuentes (`@CsvSource`, `@MethodSource`, `@CsvFileSource`). También introduce la idea de *invariante de representación* y el método `repOK()` como oráculo interno, aplicado a dos tipos de datos abstractos clásicos: una pila acotada (`StackAr`) y una cola circular acotada (`BoundedQueue`). Por último, se rescata un caso real de defecto, el *Zune bug*, como excusa para practicar debugging guiado por tests.

El práctico se apoya en los conceptos del capítulo 3 del libro de Ammann y Offutt, en particular en el patrón *arrange-act-assert* y en la idea de definir criterios de cobertura y oráculos de manera explícita. Para acompañar el estudio, se incluye un resumen teórico separado (`resumen_teorico_tp2.tex`) basado en la carpeta `material/`.

A lo largo del documento se asume que el lector está familiarizado con los conceptos básicos vistos en el TP1: *fault*, *error*, *failure* y el modelo RIPR.

1. Ejercicio 1

1.1. Lectura

Lectura del capítulo 3 de *Introduction to Software Testing* (Ammann y Offutt, 2da edición), titulado *Test Automation*. Las ideas más relevantes para el resto del práctico son:

- Distinción entre **Test Requirement** (TR) y **Test Coverage Criterion** (TCC). Un TR es una propiedad que un test debe satisfacer; un TCC es una regla que, dado un artefacto bajo prueba, genera el conjunto de TRs a cubrir.
- Noción de **subsumption** entre criterios: C_1 subsume a C_2 si todo test set que cubre C_1 también cubre C_2 .
- Patrón **arrange / act / assert** para tests automatizados, separación de *fixtures*, drivers y stubs.
- Idea de *Test-Driven Development* como técnica de diseño.
- Marco **Model-Driven Test Design** (MDTD): aplicar los mismos criterios a cuatro modelos — control flow, data flow, logic, input space — usando una capa de abstracción intermedia.

El resumen teórico que acompaña este informe (sección final del repositorio) reorganiza estas ideas con más detalle.

2. Ejercicio 2

2.1. Objetivo

Reescribir los tests del ejercicio 3 del Práctico 1 como tests parametrizados sobre la clase

`assignment2_exercises.SimpleRoutines`

La consigna exige usar al menos un `@MethodSource` y al menos un caso en formato CSV (`@CsvSource` o `@CsvFileSource`).

2.2. Estrategia de parametrización

Cada método se prueba con una fuente distinta, lo que permite ilustrar las tres alternativas principales que provee JUnit 5:

- `findLast`: `@CsvSource` *inline* en el código.
- `lastZero`, `countPositive`: `@MethodSource`, porque conviene devolver `int[]` directamente.
- `oddOrPos`: `@CsvFileSource`, leyendo el archivo `odd_or_pos_cases.csv` desde `src/test/resources`.

2.3. Casos elegidos y modelo RIPR

La selección de casos sigue el análisis del TP1. Para cada uno de los cuatro programas se incluyen tanto un caso (c) — ejecuta el defecto pero no produce falla — como un caso (d) — ejecuta el defecto y produce falla observable. Para `countPositive` y `oddOrPos` se incluye también el caso (b) (arreglo vacío), donde el defecto ni siquiera se alcanza.

Programa	Caso (c) (pasa)	Caso (d) (revela el defecto)
<code>findLast</code>	<code>[5,2,3], 2 → 1</code>	<code>[2,3,5], 2 → 0</code>
<code>lastZero</code>	<code>[1,0,2] → 1</code>	<code>[0,1,0] → 2</code>
<code>countPositive</code>	<code>[-4,2,2] → 2</code>	<code>[-4,2,0,2] → 2</code>
<code>oddOrPos</code>	<code>[2,4,-2] → 2</code>	<code>[-3,-2,0,1,4] → 3</code>

2.4. Implementación abreviada

```
1 @ParameterizedTest(name = "findLast({0}, {1}) -> {2}")
2 @CsvSource({
3     "'5,2,3', 2, 1",
4     "'2,3,5', 2, 0"
5 })
```

```

6 void findLast_casosDelEjercicio3(String valores, int objetivo,
  int esperado) {
7     assertEquals(esperado, SimpleRoutines.findLast(parseArray(
      valores), objetivo));
8 }
9
10 @ParameterizedTest(name = "lastZero({0}) -> {1}")
11 @MethodSource("lastZeroCases")
12 void lastZero_casosDelEjercicio3(int[] entrada, int esperado) {
13     assertEquals(esperado, SimpleRoutines.lastZero(entrada));
14 }
15
16 @ParameterizedTest(name = "oddOrPos({0}) -> {1}")
17 @CsvFileSource(resources = "/assignment2_exercises/
  odd_or_pos_cases.csv", numLinesToSkip = 1)
18 void oddOrPos_casosDelEjercicio3(String valores, int esperado) {
19     assertEquals(esperado, SimpleRoutines.oddOrPos(parseArray(
      valores)));
20 }

```

2.5. Resultado sobre la versión defectuosa

La clase `SimpleRoutines` provista por la cátedra para este práctico es la versión *con* los cuatro defectos del TP1. Cuando se ejecuta la suite parametrizada se obtienen exactamente cuatro *failures*, uno por programa, correspondientes a los casos (d):

- `findLast([2,3,5], 2)` devuelve -1 en lugar de 0 (la guarda `i > 0` omite el índice cero).
- `lastZero([0,1,0])` devuelve 0 en lugar de 2 (recorre de izquierda a derecha y se queda con el primer cero).
- `countPositive([-4,2,0,2])` devuelve 3 en lugar de 2 (la condición `>= 0` cuenta el cero como positivo).
- `oddOrPos([-3,-2,0,1,4])` devuelve 2 en lugar de 3 (la condición `% 2 == 1` no detecta impares negativos en Java).

Esto demuestra que la nueva suite parametrizada preserva la capacidad de detección que ya tenía el conjunto de tests no parametrizado del TP1.

2.6. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true  
-Dtest=SimpleRoutinesParameterizedTest test
```

3. Ejercicio 3

3.1. Objetivo

Construir una suite completa para `StackAr`, con fixture compartido vía `@BeforeEach`, marcando explícitamente las secciones *arrange*, *act* y *assert* y agregando al menos dos tests negativos. Completar `repOk()` con el invariante de representación pedido y testearlo.

3.2. Modelo de la clase

`StackAr` es una pila acotada implementada sobre un arreglo `elems` con un puntero `sp` al tope, donde:

- $sp = -1$ representa la pila vacía;
- las posiciones $0, 1, \dots, sp$ contienen elementos válidos;
- las posiciones $sp + 1, \dots, elems.length - 1$ deben estar libres (`null`).

3.3. Implementación de `repOk`

El invariante pedido se traduce naturalmente en cuatro chequeos:

```
1 public boolean repOk(){  
2     if (elems == null) {  
3         return false;  
4     }  
5     if (sp < -1 || sp >= elems.length) {  
6         return false;  
7     }  
8     for (int i = 0; i < elems.length; i++) {  
9         if (i > sp && elems[i] != null) {  
10             return false;  
11         }  
12         if (i <= sp && elems[i] == null) {  
13             return false;  
14         }  
15     }  
16     return true;
```

3.4. Correcciones sobre la implementación

La construcción del invariante puso al descubierto dos problemas en el código provisto:

1. **pop() no respetaba LIFO.** La versión original devolvía el nuevo tope (es decir, el elemento que quedaba después de decrementar `sp`) en lugar del elemento desapilado, y además no limpiaba la celda liberada. Esto, además de ser semánticamente incorrecto, violaba la cláusula “ $\forall i > sp : \text{elems}[i] = \text{null}$ ”. La versión corregida guarda el tope, lo limpia y decrementa el puntero:

```
1 public Object pop() throws IllegalStateException {
2     if (isEmpty()) throw new IllegalStateException();
3     Object rv = this.elems[sp];
4     this.elems[sp] = null;
5     sp--;
6     return rv;
7 }
```

2. **push aceptaba null.** La cláusula “ $\forall i \leq sp : \text{elems}[i] \neq \text{null}$ ” es incompatible con permitirlo. Se agregó una validación que rechaza `null` con `IllegalArgumentException`.

3.5. Suite de tests

Se cubren todos los métodos públicos. Cada test marca con comentarios el patrón *arrange / act / assert*. Algunos ejemplos representativos:

```
1 @BeforeEach
2 void setUp() {
3     // Todas las pruebas parten de una pila acotada de capacidad
4     // 3.
5     stack = new StackAr(3);
6 }
7 @Test
8 void popDebeRetornarUltimoElementoYReducirSize() {
9     // arrange
10    stack.push("A");
11    stack.push("B");
12    // act
13    Object valor = stack.pop();
14    // assert
```

```

15     assertEquals("B", valor);
16     assertEquals(1, stack.size());
17     assertEquals("A", stack.top());
18 }
19
20 @Test
21 void pushConNull_debeLanzarExcepcion() {
22     assertThrows(IllegalArgumentException.class, () -> stack.push
23         (null));
24 }

```

Los tests negativos cubren al menos cuatro escenarios: `push` en pila llena, `push(null)`, `pop` en pila vacía y `top` en pila vacía.

3.6. Tests de `repOk` y uso de reflexión

Para probar las ramas `false` de `repOk` no alcanza la API pública: la clase no permite, por ejemplo, dejar `sp` con un valor imposible o un hueco en la zona activa. Se usó `java.lang.reflect.Field` para corromper el estado interno de forma controlada. Esta es una técnica que el libro discute al hablar de *controllability*: cuando el código bajo prueba esconde su estado, hace falta un mecanismo adicional para forzar las situaciones que queremos verificar.

```

1 @Test
2 void repOkConHuecoEnZonaActiva_debeRetornarFalse() throws
3     Exception {
4     stack.push("A");
5     stack.push("B");
6     Object[] elems = getElems(stack);
7     elems[1] = null; // creamos un hueco
8     assertFalse(stack.repOk());
9 }

```

3.7. Inconvenientes encontrados

- No se puede romper el invariante desde la API pública sin recurrir a reflexión: para algunos casos negativos hizo falta tocar campos privados.
- Implementar `repOk` expuso el problema de `pop`, que no era visible mientras los tests sólo miraban el resultado del método aislado.
- `equals` usa la longitud del arreglo completo, así que dos pilas con misma capacidad y

mismos elementos son iguales, pero dos pilas con distinta capacidad y mismos elementos no lo son. Es coherente con el modelo de pila acotada, pero conviene tenerlo presente al diseñar tests.

3.8. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=StackArTest test
```

4. Ejercicio 4

4.1. Objetivo

Proveer tests parametrizados en formato CSV para el método `Min.min(List<? extends T>)`. Agregar tests negativos para las tres excepciones documentadas en el Javadoc: `IllegalArgumentException`, `NullPointerException` y `ClassCastException`.

4.2. Estructura de los archivos CSV

Se eligió codificar cada lista como un único string con elementos separados por punto y coma. Esto permite tener una sola columna “valores” por caso y mantener el CSV legible:

```
# min_valid_cases.csv
valores,esperado
"3;2;1",1
"5",5
"-2;4;0",-2
"7;7;9",7
"10;-1;3;-1",-1
```

```
# min_invalid_cases.csv
valores,excepcion
"EMPTY",IllegalArgumentException
"null;2;3",NullPointerException
"1;null;3",NullPointerException
```

4.3. Mapeo CSV → ejecución

- `EMPTY` representa una lista vacía.
- Un token `null` dentro de la lista se traduce a un elemento nulo de la `List<Integer>`.
- El nombre de la excepción esperada se mapea a `Class<? extends Throwable>` en el propio test, manteniendo el CSV libre de detalles del lenguaje.

4.4. Tests parametrizados

```
1 @ParameterizedTest(name = "min({0}) -> {1}")
2 @CsvFileSource(resources = "/assignment2_exercises/
   min_valid_cases.csv", numLinesToSkip = 1)
3 void min_conCasosValidosCsv_debeRetornarMinimo(String valores,
   int esperado) {
4     List<Integer> lista = parseIntegerList(valores);
5     assertEquals(esperado, Min.min(lista));
6 }
7
8 @ParameterizedTest(name = "min({0}) lanza {1}")
9 @CsvFileSource(resources = "/assignment2_exercises/
   min_invalid_cases.csv", numLinesToSkip = 1)
10 void min_conCasosInvalidosCsv_debeLanzarExcepcion(String valores,
   String excepcionEsperada) {
11     List<Integer> lista = parseIntegerList(valores);
12     assertThrows(exceptionType(excepcionEsperada), () -> Min.min(
        lista));
13 }
```

4.5. Tests negativos adicionales

Dos escenarios no se prestan a representación CSV y se codifican como `@Test` clásicos:

- Lista null (no es lo mismo que una lista vacía).
- Lista heterogénea con elementos no mutuamente comparables. Se usa un `raw type` para inyectar la mezcla `[1, "dos"]` sin que el compilador rechace el código:

```
1 @Test
2 @SuppressWarnings({"rawtypes", "unchecked"})
3 void min_conElementosNoComparables_debeLanzarClassCastException()
4 {
5     List lista = Arrays.asList(1, "dos");
6     assertThrows(ClassCastException.class, () -> Min.min(lista));
7 }
```

4.6. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true
    -Dtest=MinCsvParameterizedTest test
```

5. Ejercicio 5

5.1. Objetivo

Escribir tests parametrizados para `ZuneBug.currentYear(int days)`. Si los tests revelan alguna falla, hacer debugging y corregir el defecto.

5.2. Contexto - el Zune bug

ZuneBug reproduce un defecto real: el firmware del Zune calculaba el año actual a partir de los días transcurridos desde el 1 de enero de 1980. El 31 de diciembre de 2008 (año bisiesto) todos los dispositivos colgaron al entrar al bucle por la combinación de `days = 366` y `year = 2008`. El defecto era una combinación de *off-by-one* en la guarda y una rama bisiesta sin `else`.

5.3. Versión original

```
1 public static int currentYear (int days) {
2     int year = 1980;
3     while (days > 365) {
4         if (isLeapYear(year)) {
5             if (days > 366) {
6                 days -= 366;
7                 year += 1;
8             }
9             // sin else: si cae en este else implicito,
10            // ni days ni year se actualizan -> loop infinito
11        } else {
12            days -= 365;
13            year += 1;
14        }
15    }
16    return year;
17 }
```

5.4. Diseño de los tests parametrizados

Se construyeron dos tests complementarios:

1. `casosDeBorde` con `@MethodSource`, cubriendo valores específicos elegidos por análisis de bordes: 1, 365, 366, 367, 731 y 1461.

2. `currentYear_debeCoincidirConOracle` con `@ValueSource`, que compara la rutina contra `ZuneBug.oracle`, una implementación de referencia basada en `LocalDate.plusDays`. Esto introduce la idea de *differential testing* contra un oráculo confiable.

Ambos tests envuelven la ejecución en una llamada del estilo `assertTimeout-Preemptively(Duration.ofMillis(200))` para que un eventual loop infinito se traduzca en un *failure* y no en un *hang*.

5.5. Fallas detectadas

- **Loop infinito** para `days = 366`: la guarda exterior `days > 365` se cumple, pero la rama bisesta solo descuenta cuando `days > 366`. Como no hay `else`, el cuerpo del `while` no progresa.
- **Off-by-one** en transiciones exactas, por ejemplo `days = 731` y `days = 1461`: el método retornaba un año menos del que indica el oráculo.

5.6. Corrección aplicada

Se reescribió el bucle haciendo explícito cuántos días tiene el año que se está consumiendo:

```
1 public static int currentYear (int days) {
2     int year = 1980;
3     while (true) {
4         int daysInYear = isLeapYear(year) ? 366 : 365;
5         if (days >= daysInYear) {
6             days -= daysInYear;
7             year += 1;
8         } else {
9             break;
10        }
11    }
12    return year;
13 }
```

Mientras queden días suficientes para completar el año en curso, se descuentan y se avanza. Cuando no, se devuelve el año actual. La condición `>=` maneja correctamente los bordes y la rama del bucle siempre progresa, lo que elimina el loop infinito.

5.7. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true  
-Dtest=ZuneBugParameterizedTest test
```

Luego de la corrección la suite reporta Tests run: 21, Failures: 0, Errors: 0.

6. Ejercicio 6

6.1. Objetivo

Implementar `repOK()` para `BoundedQueue` y verificar mediante tests parametrizados que el invariante se mantiene tras secuencias de `enqueue` y `dequeue`.

6.2. Modelo de la clase

`BoundedQueue` es una cola FIFO acotada, implementada como buffer circular. Los atributos son `elements` (arreglo), `size`, `front` (índice del próximo a desencolar), `back` (índice donde se escribe el próximo `enqueue`) y `capacity`.

6.3. Invariante de representación

Las cláusulas que valida `repOK` son:

1. `elements != null` y `elements.length == capacity`.
2. `capacity >= 0`.
3. $0 \leq \text{size} \leq \text{capacity}$.
4. Caso `capacity = 0`: `size`, `front` y `back` valen cero.
5. $0 \leq \text{front} < \text{capacity}$ y $0 \leq \text{back} < \text{capacity}$.
6. **Buffer circular**: `back == (front + size) % capacity`.
7. Zona ocupada sin huecos: para todo $i \in [0, \text{size})$, `elements[(front+i) % capacity] != null`.
8. Zona libre limpia: el resto de las posiciones son `null`.

6.4. Implementación

```
1 public boolean repOK() {  
2     if (elements == null) return false;
```

```

3      if (capacity < 0 || elements.length != capacity) return false
        ;
4      if (size < 0 || size > capacity) return false;
5      if (capacity == 0) return size == 0 && front == 0 && back ==
        0;
6      if (front < 0 || front >= capacity || back < 0 || back >=
        capacity) return false;
7      if (back != (front + size) % capacity) return false;
8      boolean[] occupied = new boolean[capacity];
9      for (int i = 0; i < size; i++) {
10         int idx = (front + i) % capacity;
11         occupied[idx] = true;
12         if (elements[idx] == null) return false;
13     }
14     for (int i = 0; i < capacity; i++) {
15         if (!occupied[i] && elements[i] != null) return false;
16     }
17     return true;
18 }

```

6.5. Tests parametrizados

Para no escribir un test por cada secuencia se utilizó un mini-DSL textual: **E:valor** representa **enqueue(valor)** y **D** representa **dequeue()**. Una cadena como **E:a;E:b;D;E:c** encadena las operaciones, y un método **apply** las ejecuta una por una, chequeando **repOK** en cada paso.

Escenarios válidos (@MethodSource)

Se cubren situaciones representativas: llenado y vaciado completo, wrap-around (cuando **back** da la vuelta al final del arreglo), capacidades chicas (1 y 2) donde la aritmética modular es más delicada.

Capacidad	Operaciones	Cola final
3	E:a;E:b;D;E:c	[b, c]
3	E:x;E:y;D;E:z;E:w	[y, z, w]
4	E:1;E:2;E:3;D;D;E:4;E:5	[3, 4, 5]
2	E:p;E;q;D;E:r	[q, r]
1	E:solo;D	[]

Escenarios inválidos

Las operaciones inválidas (`deQueue` en vacía, `enQueue(null)`, `enQueue` en llena) deben lanzar la excepción documentada y, además, el invariante debe seguir cumpliéndose después del error: una operación que falla no debe corromper la cola.

```
1 @ParameterizedTest(name = "cap={0}, setup={1}, fail={2}, ex={3}")
2 @MethodSource("invalidScenarios")
3 void repOK_debeMantenerseCuandoUnaOperacionInvalidaLanzaExcepcion
4 (
5     int capacity, String setupOperations,
6     String failingOperation, Class<? extends Throwable>
7     expectedException
8 ) {
9     BoundedQueue queue = new BoundedQueue(capacity);
10    for (String op : splitOperations(setupOperations)) {
11        apply(queue, op, new ArrayList<>());
12        assertTrue(queue.repOK());
13    }
14    assertThrows(expectedException, () -> apply(queue,
15        failingOperation, new ArrayList<>()));
16    assertTrue(queue.repOK());
17 }
```

6.6. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true
-Dtest=BoundedQueueParameterizedTest test
```

Conclusiones

El práctico 2 consolida las herramientas introducidas en el TP1 y agrega dos ejes nuevos:

- **Data-driven testing** con JUnit 5: separar los datos de los casos del cuerpo del test reduce la duplicación y mejora la legibilidad. `@CsvSource` es práctico para casos chicos e *inline*, `@CsvFileSource` permite mantener los datos como recurso externo y `@MethodSource` es el camino natural cuando los argumentos no son primitivos o cadenas simples.
- **Invariante de representación como oráculo interno**: implementar `repOk` para `StackAr` y `BoundedQueue` expone defectos que los tests funcionales por sí solos no detectan. Es una técnica especialmente valiosa para clases con estado.

El caso del *Zune bug* ilustra un patrón habitual cuando se hace debugging guiado por tests: los casos elegidos por análisis de bordes (`days = 366`) y el contraste contra un oráculo confiable (`LocalDate.plusDays`) bastan para localizar y luego corregir un defecto sutil. Para no quedarse esperando un cuelgue, conviene envolver la ejecución con un timeout preemptivo.